TECHNICAL APPENDIX

FOR THE INTERVIEWER WHO WENT PAST PAGE 10. RESPECT.

A. The hash chain, precisely — construction, properties, and honest limits

agent/trace.py — the construction

```
step_record = {hypothesis, sql, statistic, verdict, ts}
canonical   = json.dumps(step_record, sort_keys=True)
step_hash   = sha256(prev_hash + canonical).hexdigest()
chain.append({**step_record, "hash": step_hash})
# verify: replay the fold; any edit ⇒ every later hash mismatches
```

Guarantees: integrity after sealing (tamper-evidence) and *reproducibility* — each step embeds its literal SQL and statistics, so a verifier can re-run every query against the same dataset and reproduce the verdicts.
Non-guarantees, stated before you ask: execution attestation (mitigated by re-runnability), author authentication (production answer: signing keys), and storage-level rewrite of the entire chain plus root (production answer: anchor the root hash externally). Tamper tests in CI attempt forgery on every commit.

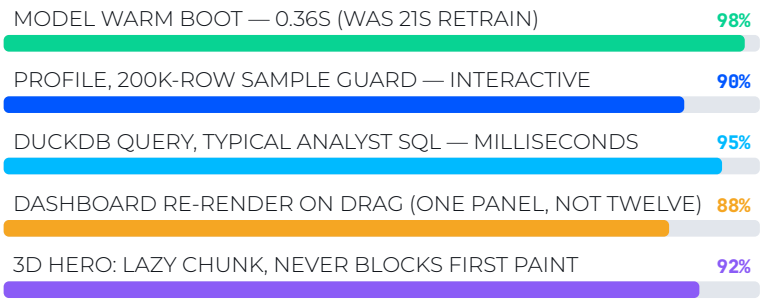
B. Risk model card — the honest one-pager regulators wish every model had

`data`	ULB credit-card fraud, 284,807 transactions, 0.17% positive class; synthetic-but-labeled fallback when absent (reported in UI)
`algorithm`	GradientBoostingClassifier + IsolationForest (anomaly stream)
`features`	8 engineered: amount_log, hour, velocity, channel/geo/kyc risk encodings, account age
`split`	stratified hold-out; **no metric is ever reported on training data**
`metrics`	ROC-AUC 0.913 · PR-AUC 0.65 (the honest one under imbalance)
`threshold`	user-tunable, live precision/recall trade-off; a policy choice, deliberately not hardcoded
`persistence`	joblib artifact + metadata; warm boot 0.36s (vs 21 s retrain)
`known limits`	global (not per-tenant) model; no drift monitoring yet; temporal split is the named V2 upgrade for production realism

C. SQL sandbox defense map — four layers, one scar

L1 — engine: DuckDB `enable_external_access=False` — filesystem unreachable even if all other layers fail. The load-bearing wall.
L2 — guard: SELECT-only allowlist · comment rejection (smuggling vector) · catalog/schema-probe denial · table-function blocklist (`read_csv` et al.).
L3 — resources: result-row cap, execution timeout, per-IP rate limit upstream.
L4 — verification: adversarial corpus in CI; grew by one entry the day it caught a real file-read vulnerability in my own guard. The attack is now a permanent regression test. Enumerating badness fails; removing capability works.

D. Performance ledger — measured, not vibes



Known ceilings, in failure order under load: synchronous profiling → dataset-cache memory → DuckDB concurrency. Fix order: async job queue → LRU + spill → per-user query caps. Postgres yawns at 10x. Full reasoning: war-room Q41–Q48.

E. Test inventory (82 total) — where the paranoia concentrates

`api contracts (pytest)`	every endpoint: shape, status, unhappy paths, auth, rate limits
`profiling edge cases`	mixed-format dates, numeric IDs, constant columns, all-null columns
`adversarial SQL corpus`	injection museum: comments, catalogs, file functions, stacked queries — 1 real catch
`hash-chain tamper suite`	forge a step, reorder steps, truncate chain — all must be detected
`metrics discipline`	asserts held-out evaluation; the test that would catch a training-set swap
`frontend (vitest)`	chart options logic, error boundaries, insight panel rendering

F. V2, with receipts — already in requirements.txt, not in slides

SHAP per-prediction explanations — completes show-your-work at the model layer; waterfall panel designed for the Risk page.
XGBoost + LightGBM + Optuna — the model tournament: same honest backtest selection as forecasting, stronger competitors, tuned not hand-set.
Async ingestion — queue + progress events; kills the file-size ceiling.
Hypothesis (property-based testing) — fuzz the profiler and the SQL guard with generated adversaries; paranoia, industrialized.

> Standing offer: pick any subsystem on these four pages and ask for the whiteboard version. The appendix exists because "can you go deeper?" deserves "yes" in writing.